CS6868 Research Mini-Project

Project 23: A Lock-Free Synchronous Dual Queue in OCaml 5

Rohan Kadam
cs22b083@smail.iitm.ac.in

Bhargav
cs22b087@smail.iitm.ac.in

Venkat
cs22b027@smail.iitm.ac.in

May 1, 2026

Abstract

We implemented a lock-free synchronous dual queue based on the design of Scherer, Lea and Scott [3], together with two baselines: a blocking mutex/condition-variable synchronous queue, and a single-slot lock-free exchanger from Lecture 08. All three were written in OCaml 5 using the new `Atomic` module and `Domain` parallelism. We verified correctness using manual concurrent stress tests and a QCheck-Lin linearizability test on the blocking baseline, and we ran the lock-free implementation under ThreadSanitizer (TSAN) on a TSAN-enabled OCaml switch. Throughput was measured in matched pairs per second across 2–8 domains under balanced, enqueue-heavy and dequeue-heavy workloads. The lock-free dual queue beats the blocking baseline by roughly 4–10× across every tested configuration, including the imbalanced workloads, so the reservation-node design is a clear win over mutex/condition variables at our scales. The exchanger, which only handles paired threads in the balanced configuration, adds a further 1.2–2.9× on top of the dual queue and, unlike the queue, its throughput scales near-linearly with the domain count. This suggests that elimination on paired arrivals is a worthwhile optimisation to layer on top of the dual queue, which we discuss as future work.

1 Goals

We picked Project 23 (Synchronous Dual Queue) because the algorithm neatly combines two ideas the course covered separately: the Michael–Scott lock-free queue and the use of CAS to coordinate handoff between threads (Lecture 08, exchanger and elimination array). A standard concurrent queue is asynchronous: an enqueuer always succeeds, even if no dequeuer is around. A *synchronous* queue, in contrast, requires a rendezvous: an `enqueue` must wait until some `dequeue` actually receives the value, and vice versa. Java's `SynchronousQueue` and Go channels of capacity zero are real systems that work this way. The *dual queue* of Scherer, Lea and Scott [3] unifies both modes by allowing reservation nodes to live in the same Michael–Scott structure, with producers fulfilling consumer reservations and vice versa.

The connection to course material is direct: the dual queue is the Michael–Scott queue from Lecture 08 plus condition synchronization done through CAS rather than condition variables, which is exactly what Scherer and Scott call "nonblocking concurrent data structures with condition synchronization."

Research question. *How does the lock-free dual queue's rendezvous throughput compare against a mutex/condition-based synchronous queue, and does the overhead of reservation nodes pay off at higher thread counts?*

2 Background

Synchronous queues. A synchronous queue has two operations, `put` and `take`, both of which block until they meet a partner of the opposite kind. Conceptually it is a buffer of capacity zero: every value is handed off directly from a producer to a consumer with no storage in between. The classical implementation in textbooks [1, §10.6.1] uses a mutex and two condition variables, one for waiting producers and one for waiting consumers, and every `put`/`take` pair causes two context switches.

Dual queue intuition. The Scherer–Lea–Scott dual queue avoids the lock by storing the "waiting" state inside the queue itself. The queue holds nodes of two kinds:

- `Data`: a producer is waiting; the node carries `Some v`.
- `Request`: a consumer is waiting; the node's value slot is initially `None`.

The key invariant is that all live nodes in the queue have the *same* kind — the queue is either empty, all-Data, or all-Request, never mixed. An arriving thread inspects the head:

- If the queue is empty or only contains waiters of the same kind as the arriving thread, the thread appends a reservation (using a Michael–Scott-style two-step tail CAS) and spins until its own node is fulfilled.
- Otherwise, the head holds an opposite-kind reservation; the arriving thread *fulfils* that reservation by CAS-ing the payload slot from its waiting state to its fulfilled state, then helps advance the head.

The CAS that flips the payload slot is the rendezvous: it is the linearization point for both the fulfilling thread and the waiting thread, simultaneously. After the CAS, the waiter sees its node is fulfilled and returns; the fulfiller helps advance `head` and also returns. Listing 1 sketches the producer side.

Listing 1: Producer (`put`) sketch.

```
let rec put q value =
  help_advance_head q;
  match Atomic.get (Atomic.get q.head).next with
  | Some ({ kind = Request; item; _ } as r) ->
      (* Consumer is waiting: fulfil the head request. *)
      if Atomic.compare_and_set item None (Some value) then
        ignore (Atomic.compare_and_set q.head ... r)
      else put q value
  | _ ->
      (* Empty or only producers waiting: append reservation and spin. *)
      let node = {kind=Data; item=Atomic.make (Some value); next=Atomic.make None} in
      match try_append_node q node with
      | Appended -> wait_until_fulfilled node
      | Retry    -> put q value
```

Linearizability. A history is linearizable if every operation appears to take effect at a single instant between its invocation and response. For our dual queue, that instant is the successful payload CAS at the rendezvous. Both `put` and `take` share the same linearization point: this is the elegant part of the algorithm and the part that took us the longest to internalise.

3 Tasks Undertaken

3.1 Implementation

We implemented three handoff primitives, all in OCaml 5 using `Atomic.t` and `Domain.spawn`. Source paths below are relative to the repository root.

- **Lock-free synchronous dual queue** (`code/lockfree_sync_dual_queue.ml`, 166 LoC). Nodes carry an `'a option Atomic.t` payload so the same atomic flips represent both the value and the fulfilment state. We use `Atomic.make_contended` for `head`, `tail` and every node's `item` slot to push them onto separate cache lines and reduce false sharing. `help_advance_head` is called eagerly at the top of `put`/`take` so that fulfilled nodes do not pile up at the front.
- **Single-slot exchanger** (`code/lockfree_exchanger.ml`, 61 LoC). One `state Atomic.t` cycles through `Empty`, `Waiting v`, `Busy v`. The first arrival CASes `Empty` → `Waiting v` and spins; the second arrival CASes `Waiting v` → `Busy v'` and returns. The successful second CAS is the linearization point for both threads.
- **Mutex + condition baseline** (`code/lock_sync_dual_queue.ml`, 107 LoC). A single mutex protects two `Stdlib.Queue` structures of waiting producers and waiting consumers, each carrying its own `Condition.t`. We also expose non-blocking `try_put` and `try_take` variants that we use for QCheck-Lin testing (the blocking versions cause QCheck workers to deadlock).

Subtle invariants. The dual queue has an invariant we got wrong twice during development: the queue may only contain reservations *of one kind at a time*. The first version of `put` appended a Data node whenever it saw a non-empty queue, which broke the invariant on a tail-CAS race and caused two consumers to mis-pair with the same producer in stress tests. Our fix is the conjunctive guard

```
if last == first || last.kind = node.kind then ...
```

which mirrors how the original paper splits the head-fulfil and tail-append cases. We also re-read `q.tail` after the snapshot to detect concurrent appends, in the same way Michael–Scott re-validates `tail`.

Two-step tail. We deliberately kept the two-step tail update (set `last.next` first, then CAS `q.tail`) because it lets a stalled appender hand off the rest of its work to the next thread, which is essential for lock-freedom in OCaml's runtime where any `Domain` can be descheduled.

3.2 Testing and verification

Manual stress tests (`code/test_manual.ml`). We wrote five concurrent tests, each exercising a different configuration: 1p/1c single pair, 1p/1c repeated pairs (10 and 100), 4p/4c with 200 pairs and a per-value duplicate detector, plus exchanger tests for 2, 4 and 8 domains. The 4p/4c test was the most useful: each producer publishes values $\{i, i+P, i+2P, \dots\}$ where P is the producer count, the consumers record receipts in a shared array guarded by a mutex, and the test fails if any value is missed, duplicated or out of range. This caught the same-kind invariant bug mentioned above.

QCheck-Lin (`code/qcheck_lin.ml`). We use QCheck-Lin's `Lin_domain` backend with the `multicoretests` library. Because the blocking variants of `put`/`take` would simply deadlock the QCheck-Lin workers (there is no consumer when QCheck-Lin tries to linearise a single producer's

history), we expose `try_put`/`try_take` on the lock baseline and exercise those instead. QCheck-Lin runs 500 random concurrent histories per invocation and checks that each one is sequentially consistent against an explicit "either rendezvous now or fail" specification. All 500 traces passed.

TSAN. We built the project on a TSAN-enabled OCaml switch (an `options+tsan` variant of OCaml 5) and re-ran `test_manual` and a 1 000-pair benchmark (`results/benchmark-tsan.csv`). TSAN reported no data races on the lock-free queue. As a sanity check we also wrote a deliberately broken non-atomic version (`code/non_atomic_lockfree_sync_dual_queue_tsan.ml`) that uses plain `ref` cells for the same fields, and confirmed that TSAN does flag races there. This convinced us that the TSAN setup was actually instrumenting the code rather than silently passing.

Producer–consumer pipeline (`code/producer_consumer_pipeline/pipeline_demo.ml`). A small demo with 4 producers, 8 consumers and 10 jobs per producer. The main thread spawns all consumers, spawns all producers, joins the producers, and then sends one `Stop` message per consumer through the same synchronous queue. Because every handoff is a rendezvous, the demo verifies that producers and consumers actually wait for each other rather than buffering through some implicit queue.

4 Evaluation

Setup. All measurements were taken on a Linux laptop (kernel 6.17.0-22-generic, x86_64, 12th Gen Intel Core i5-1235U, 10 cores / 12 logical threads, OCaml 5.4.0). The benchmark (`code/benchmark.ml`) spawns the requested number of producer and consumer domains, has them all spin on a shared `Atomic.bool` "go" flag, and times the wall-clock interval from setting the flag to all domains joining. Each producer `put`s a fixed share of the total pair count, each consumer `take`s its share, and we report pairs/seconds as throughput. We use 100 000 pairs per configuration, which was enough to dominate the start-up jitter on our machine. Because of the tight time budget for the project we did not sweep over multiple repetitions and did not pin domains; the trends are robust but the absolute numbers carry ±5–10% run-to-run noise, which is something we would tighten given more time.

Workloads. *Balanced* splits domains evenly between producers and consumers; *enqueue-heavy* uses $\lceil 3D/4 \rceil$ producers and the rest consumers; *dequeue-heavy* is the mirror image. Every configuration moves the same total of 100 000 matched pairs, so imbalance changes the ratio of waiters to fulfillers but not the total amount of synchronisation work.

Table 1: Throughput (pairs/s, $\times 10^3$) at 100 000 pairs. Source: `code/results/benchmark.csv`.

Workload	Implementation	2D	4D	6D	8D
Balanced	lockfree	1047.1	978.8	1242.8	1203.1
	blocking	243.6	249.0	210.2	207.3
Enqueue-heavy	lockfree	1556.9	1307.0	1313.4	1085.9
	blocking	191.0	241.9	218.8	149.0
Dequeue-heavy	lockfree	1461.2	1415.2	972.5	1069.4
	blocking	146.4	235.4	157.0	158.9

Balanced workload. On the balanced workload (Table 1) the lock-free dual queue is 4.3× faster than the blocking baseline at 2 domains, 3.9× at 4, 5.9× at 6, and 5.8× at 8. The gap is large even at 2 domains because the mutex path pays for a real `pthread_cond_wait`/`signal`

pair on every handoff, whereas the lock-free rendezvous is just two CASes and a short `cpu_relax` spin. Throughput on the lock-free queue is roughly flat in the 1.0–1.2 M pairs/s range across domain counts, which is consistent with the tail (and the head) being the serial bottleneck — adding more domains does not add more handoff parallelism when every pair still funnels through the same `head`/`tail` atomics.

Imbalanced workloads. The enqueue-heavy and dequeue-heavy columns show the same pattern: the lock-free queue stays in the 1.0–1.6 M pairs/s band, while the blocking baseline drops to 0.15–0.25 M. The lock-free advantage widens to 5–10× under imbalance. The reason is that the lock-free fast path is the same regardless of who is waiting: a fulfiller arrives, CASes the head reservation's payload slot, and leaves. The blocking implementation, by contrast, has to take the mutex, push the waiter onto the appropriate queue, release the mutex, `wait` on a condition variable, and then wake up and re-acquire when signalled — and the more often one side is waiting (which is exactly what "imbalanced" means), the more often that full wait/signal cycle runs. The 2-domain dequeue-heavy point for the blocking baseline (146 k pairs/s) is the worst of the blocking numbers, again consistent with this: a single consumer repeatedly parks on the empty queue, and every producer has to wake it.

The exchanger. The exchanger is faster than the dual queue on the balanced workload but not by as much as one might expect — 1.18× at 2 domains, 2.35× at 4, 2.90× at 6, and 2.69× at 8. Unlike the dual queue, the exchanger's throughput *does* scale with the domain count, because each pair has its own isolated `state Atomic.t` (in our harness we use a single slot, so the scaling we see is really parallelism in the two producer/consumer halves rather than across slots); with a real elimination array of D slots we would expect the gap to the queue to widen further. Either way, the fast path allocates nothing and consists of two CASes, so it is a strict lower bound on what any handoff primitive can cost. We see this as motivation for adding an elimination layer on top of the dual queue, similar to the elimination-backoff stack from Lecture 08: arrive at an exchanger slot first, only fall back to the slow Michael–Scott path if there is no partner within a budget.

Did the reservation overhead pay off? Yes, unambiguously. Across all twelve queue configurations ($\{$balanced, enqueue-heavy, dequeue-heavy$\} \times \{2, 4, 6, 8\}$ domains) the lock-free dual queue beats the mutex/condition-variable baseline by no less than 3.9× and up to 10.0×. The reservation-node cost (one extra allocation per waiting thread, plus a few atomic loads to find the queue kind) is much smaller than the cost of a kernel-level park/unpark, even on a single-machine workload with no real I/O.

5 Reflection on the Use of LLMs

We used Cursor and ChatGPT while working on this project.

6 Conclusions

Our research question was: *does the overhead of reservation nodes pay off at higher thread counts?* The answer we got is "yes, by a wide margin." Across balanced, enqueue-heavy and dequeue-heavy workloads at 2–8 domains, the lock-free dual queue gives 3.9–10.0× the throughput of the mutex/condition baseline. The win is driven by the two fast paths that the reservation design enables: an arriving thread that meets an opposite-kind waiter finishes the rendezvous in two CASes, and an arriving thread that has to wait spins on its own node rather than parking in the kernel. Both paths avoid the `pthread_cond` wake-up that dominates the blocking baseline's

cost. The only implementation whose throughput *grows* with the domain count in our data is the exchanger, which hints that the queue's shared `head`/`tail` atomics are now the bottleneck.

Limitations. Our benchmark is a single machine, single OCaml version, no domain pinning, and one repetition per point; the 5–10% run-to-run noise we observed is enough that some of the smaller differences in Table 1 (for example lockfree at 2D vs. 4D on balanced) should be read as "no measurable difference." We also did not implement the full elimination-array variant of the dual queue from Scherer–Lea–Scott, which is what would close the remaining ∼2–3× gap to the single-slot exchanger.

Future work. The two natural next steps are (i) layering the single-slot exchanger (or an elimination array of them) in front of the slow path to capture paired arrivals without queueing, and (ii) replacing our unbounded spin with bounded spin + `Domain` parking, which would let the dual queue degrade gracefully when the number of waiters blows past the number of cores. Both are well-known fixes from the literature and from Java's `SynchronousQueue` [2].

Contributions

Member	**Contribution (%)**
Rohan Kadam	34%
Bhargav	33%
Venkat	33%
Total	**100%**

References

[1] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020. Chapter 10, Section 10.6.1: A Naïve Synchronous Queue.

[2] Oracle. Java Platform SE documentation: SynchronousQueue, 2023. https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/SynchronousQueue.html.

[3] William N. Scherer III, Doug Lea, and Michael L. Scott. Scalable synchronous queues. *Communications of the ACM*, 52(5):100–108, 2009. doi:10.1145/1506409.1506431.